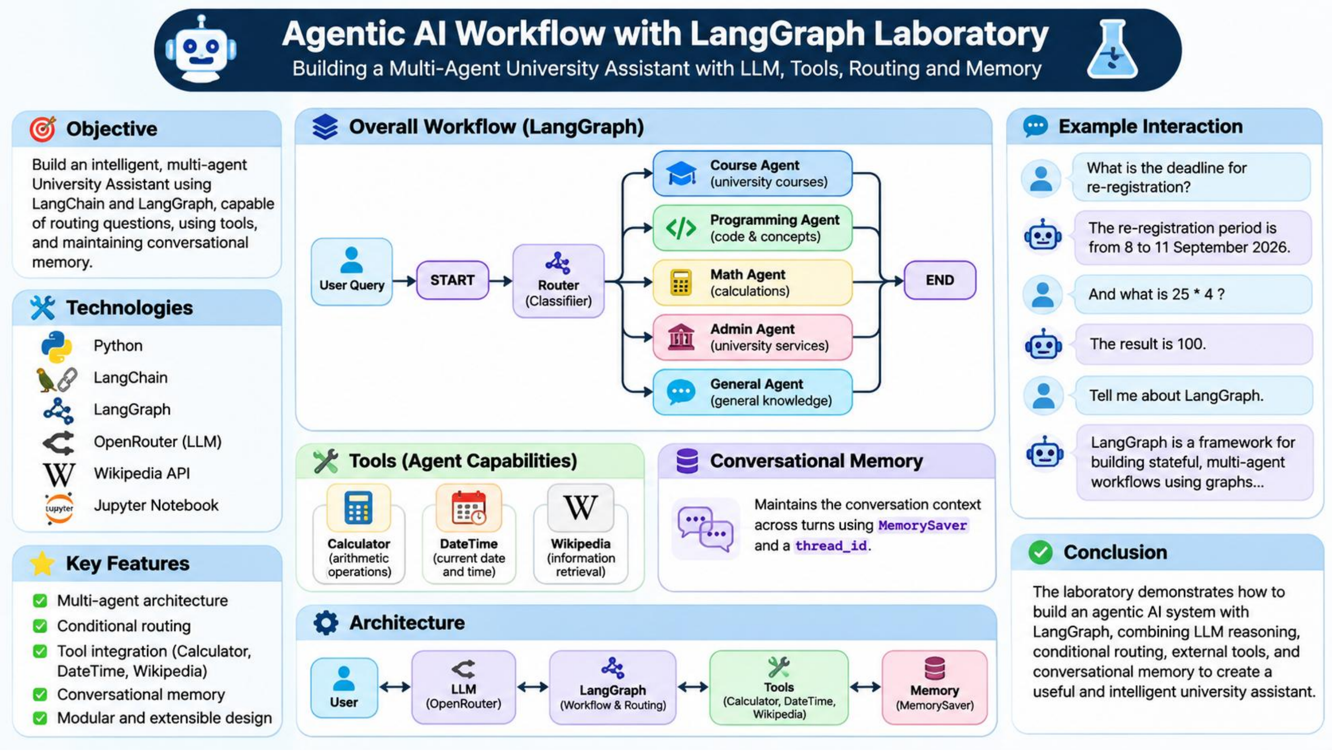
كلية العلوم و التقنيات بطنجة
ⵜⴰⵖⵉⵡⴰⵏⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⴷ ⵜⵙⵏⵉⵅⵉⵜⵉⵏ ⵏ ⵟⴰⵏⵊⴰ
Faculté des Sciences et Techniques de Tanger

جامعة عبد المالك السعدي
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵄⴰⴱⴷⵍⵎⴰⵍⴽ ⵙⵙⴰⵄⴷⵉ
Université Abdelmalek Essaâdi

Practical Lab — Building AI Agents with LangGraph



Agentic AI Workflow with LangGraph Laboratory

Building a Multi-Agent University Assistant with LLM, Tools, Routing and Memory

Objective

Build an intelligent, multi-agent University Assistant using LangChain and LangGraph, capable of routing questions, using tools, and maintaining conversational memory.

Technologies

- Python
- LangChain
- LangGraph
- OpenRouter (LLM)
- Wikipedia API
- Jupyter Notebook

Key Features

- ✓ Multi-agent architecture
- ✓ Conditional routing
- ✓ Tool integration (Calculator, DateTime, Wikipedia)
- ✓ Conversational memory
- ✓ Modular and extensible design

Overall Workflow (LangGraph)

Tools (Agent Capabilities)

Conversational Memory

Maintains the conversation context across turns using `MemorySaver` and a `thread_id`.

Architecture

Example Interaction

What is the deadline for re-registration?

The re-registration period is from 8 to 11 September 2026.

And what is 25 * 4 ?

The result is 100.

Tell me about LangGraph.

LangGraph is a framework for building stateful, multi-agent workflows using graphs...

Conclusion

The laboratory demonstrates how to build an agentic AI system with LangGraph, combining LLM reasoning, conditional routing, external tools, and conversational memory to create a useful and intelligent university assistant.

Presented by: Fatima STITOU

Supervised by: Pr. Ikram BENABDELOUAHAB
and Pr. Lotfi EL AACHAK

1. Roles of LangChain, LangGraph and an LLM

LangChain

LangChain is a framework that provides components and interfaces for developing applications powered by Large Language Models (LLMs). In this lab, it is used to connect the application to the language model, prepare model calls and define tools that can be made available to an agent.

LangGraph

LangGraph is a framework for building stateful workflows and agents represented as graphs. It organizes execution into nodes and edges, supports conditional routing between processing steps, and can integrate tools and conversation memory. In this lab, it is used to build workflows such as the question classifier, specialized agents, the tool-using agent and the university assistant.

An LLM

A Large Language Model is an AI model trained on large collections of data to understand and generate natural language. In this lab, the LLM interprets user questions, classifies them into categories, generates answers and, when tools are bound to it, can request a tool call.

2. Nodes, edges, START and END

Q1. What is a node?

A node is a processing step in a LangGraph workflow. It is generally implemented as a Python function that receives the current state, performs a task and returns state updates. Examples in the notebook include the classifier, mathematics agent, programming agent, artificial-intelligence agent and tool node.

Q2.What is an edge?

An edge connects graph nodes and defines the transition from one step to the next. A normal edge represents a fixed transition. A conditional edge calls a routing function or condition to determine which node should execute next.

Q3.What is START?

START is the special entry point of the graph. It marks the beginning of execution and connects the workflow to its first processing node.

Q4. What is END?

END is the special terminal point of the graph. Reaching END means that the current workflow execution is complete.

Q5.Why is the state passed between nodes?

The state is the shared data structure that carries information from one node to another. A node reads the values it needs and returns updates for subsequent nodes. In the first workflow of the notebook, the state contains a message and a response. In the classifier workflow, it contains the question, its category and the generated answer. This shared state allows the graph to preserve intermediate results and coordinate its different steps.

3. Conditional routing and router reliability

In Part V, the classifier asks the LLM to assign each question to one of three categories: mathematics, programming or artificial_intelligence. The routing function then selects the corresponding specialized node. In the capstone, the classifier uses five categories: COURSES, PROGRAMMING, MATHEMATICS, ADMINISTRATION and GENERAL.

A router may fail when a question is ambiguous, contains multiple topics, is outside the predefined categories, or when the LLM returns an unexpected label or additional text. The Part V routing function also sends every category that is neither mathematics nor programming to the AI branch by default. This means that an invalid or unrecognized category could be misrouted instead of being explicitly rejected. The notebook includes a GENERAL category and a fallback path in the capstone design to handle requests that do not fit a specialized route.

Routing can be made more robust through the following measures:

- **Structured output:** require the classifier to return a predictable field, such as a category, rather than unrestricted text.
- **Few-shot examples:** provide examples of questions and their expected categories in the classifier prompt.
- **Enum validation:** validate the returned category against the exact allowed labels before selecting a branch.
- **Explicit fallback:** route invalid, missing or uncertain classifications to a fallback or GENERAL node rather than silently choosing a specialized branch.
- **Evaluation**: test representative questions from every category, including ambiguous and out-of-scope questions, compare predicted and expected labels, and inspect incorrect routes.

The notebook proposes an evaluation set of six questions covering mathematics, programming and artificial intelligence, with expected categories and a routing-accuracy calculation. The actual accuracy depends on executing the notebook with the configured LLM; it is therefore not stated here as a fixed result

4. Tools and tool-using agents

A tool is a callable function that an agent can use to perform a specific operation. The notebook defines a calculator and a current date/time tool. The LLM is bound to these tools and can emit a tool call when it determines that one is needed. A conditional edge sends execution to the ToolNode when a tool call is present; after the tool executes, an edge returns control to the agent so it can produce a final answer. If no tool is required, the workflow can finish directly. The notebook explicitly warns that using Python eval() for calculator input is unsafe because it can execute arbitrary code. It therefore provides a safer calculator based on Python's AST, allowing only selected arithmetic operators and numeric constants.

5. Conversation memory

The notebook uses a messages state with the add_messages reducer, which appends new messages to the conversation history. Without a checkpointer, separate invoke calls start without the previous conversation. With MemorySaver and the same thread_id, the graph can retain conversation state between calls. A different thread_id keeps a separate conversation history.

6. Conclusion

The lab demonstrates how LangChain and LangGraph can be combined to build LLM-powered workflows. LangChain provides model and tool integration components, while LangGraph controls the execution flow through state, nodes, edges and conditional routing. Tool integration extends the agent's capabilities beyond text generation, and checkpointing enables conversational continuity. Reliable systems require validated routing, explicit fallback behavior, safe tool implementations and evaluation on representative test cases.